

Deep Learning

Lecture 11a — Reinforcement learning foundations: from imitation
to policy gradients

Why an RL lecture in a deep learning course?

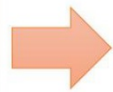
- Lectures 11–12 use RLHF, PPO, DPO and GRPO to post-train language models — this lecture builds the ideas they rest on
- Plan (following Berkeley CS285, Sergey Levine): what RL is and where it shines (robots, LLMs, image generators) → **imitation learning** and why it breaks → the **RL problem** → **policy gradients** → variance reduction, off-policy ratios, **actor-critic**, trust regions (**PPO**)
- Two bridges we will keep coming back to: **SFT is behavior cloning; GRPO is a policy gradient without a critic**
- Robotics runs through the whole story: the same recipe (imitate, then optimize) trains today's vision-language-action models

What reinforcement learning is, and where it works

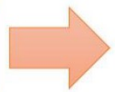
CS285 Lecture 1

What is reinforcement learning?

supervised learning



computer
program



[object label]

input: $\mathbf{x}$

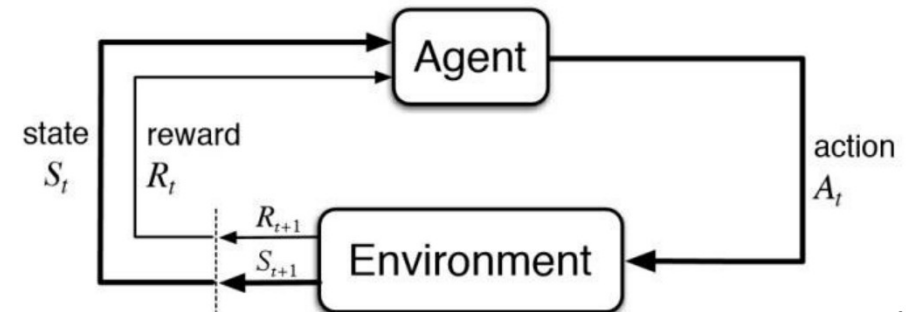
output: $\mathbf{y}$

data: $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$

goal: $f_{\theta}(\mathbf{x}_i) \approx \mathbf{y}_i$

← someone gives
this to you

reinforcement learning



input: $\mathbf{s}_t$ at each time step

output: $\mathbf{a}_t$ at each time step

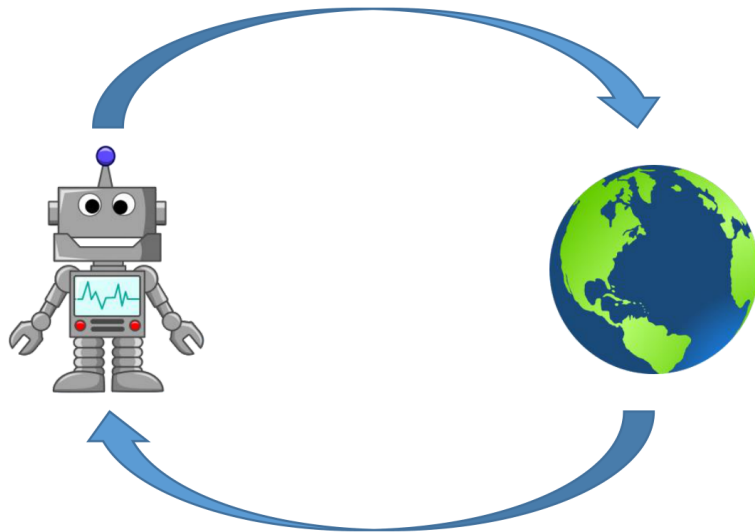
data: $(\mathbf{s}_1, \mathbf{a}_1, r_1, \dots, \mathbf{s}_T, \mathbf{a}_T, r_T)$

goal: learn $\pi_{\theta} : \mathbf{s}_t \rightarrow \mathbf{a}_t$

to maximize $\sum_t r_t$

pick your
own actions

decisions (actions)



consequences
observations (states)
rewards



Actions: muscle contractions
Observations: sight, smell
Rewards: food



Actions: motor current or torque
Observations: camera images
Rewards: task success measure (e.g., running speed)



Actions: what to purchase
Observations: inventory levels
Rewards: profit

Really complex physical tasks!



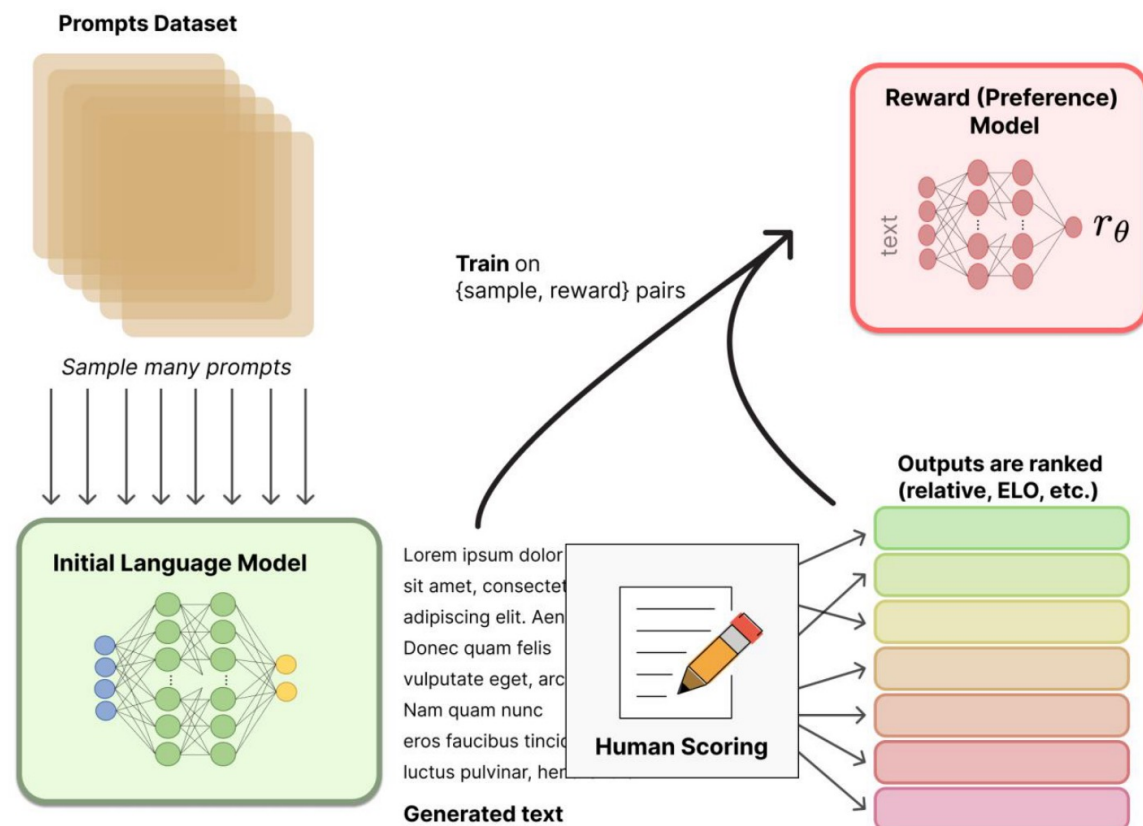
Smith et al., “Learning and Adapting Agility Skills by Transferring Experience.” 2022.

At scale in the real world

**Deep RL at Scale: Sorting Waste in Office Buildings with
a Fleet of Mobile Manipulators**

**Deep RL at Scale: Sorting Waste in Office Buildings with
a Fleet of Mobile Manipulators**

Reinforcement learning with language models



LE

Can you explain to me how RL with human feedback works for language models like I am five?

🌀

Sure! So, imagine you have a robot named Rufus who wants to learn how to talk like a human. Rufus has a language model that helps him understand words and sentences.

First, Rufus will say something using his language model. For example, he might say "I am a robot."

Then, a human will listen to what Rufus said and give him feedback on whether it sounded like a natural sentence a human would say. The human might say, "That's not quite right, Rufus. Humans don't usually say 'I am a robot.' They might say 'I'm a robot' or 'I am a machine.'"

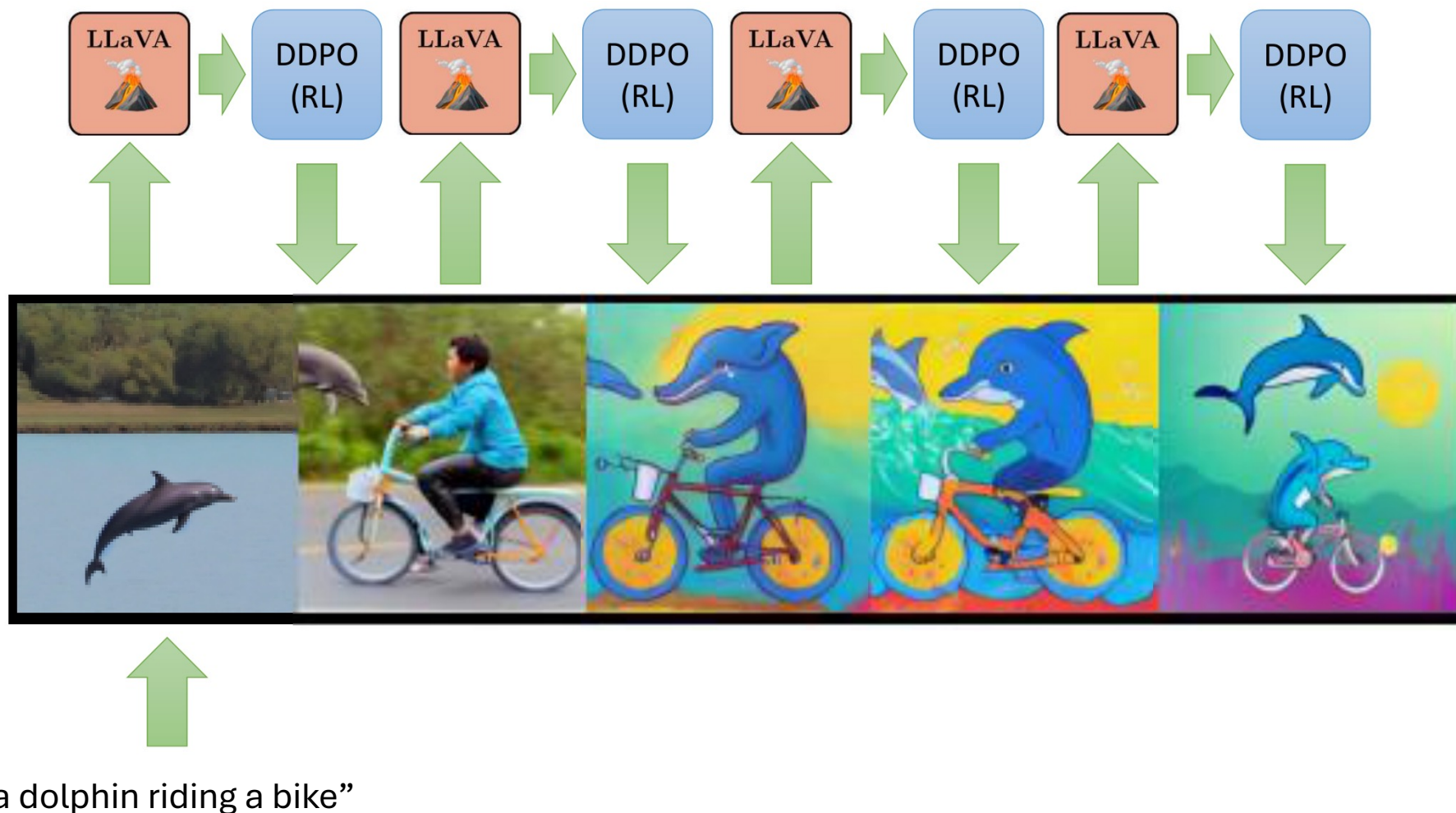
Rufus will take this feedback and use it to update his language model. He will try to say the sentence again, using the new information he received from the human. This time, he might say "I'm a robot."

The human will listen again and give Rufus more feedback. This process will continue until Rufus can say sentences that sound natural to a human.

Over time, Rufus will learn how to talk like a human thanks to the feedback he receives from humans. This is how language models can be improved using RL with human feedback.

Source: <https://huggingface.co/blog/rlhf>

Reinforcement learning with image generation



So where does that leave us?

Data-Driven AI Reinforcement Learning



Explaining a joke

Prompt

Explain this joke:

Joke: Did you see that Google just hired an eloquent whale for their TPU team? It showed them how to communicate between two

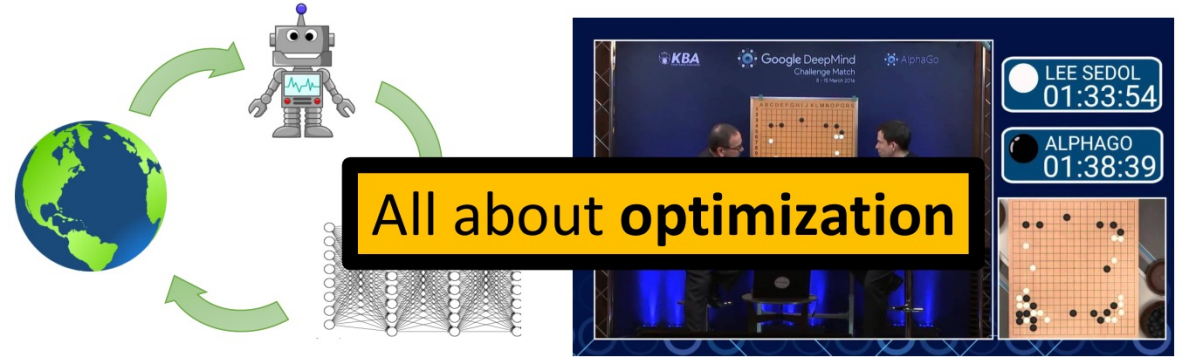
chip that Google uses TPUs. A "pod" is also a

ale is able to communicate between two groups of whales, but the speaker is pretending that the whale is able to communicate between two groups of TPUs.

All about using data

+ learns about the real world from data

- **doesn't try to do better than the data**



All about optimization

LEE SEDOL 01:33:54

ALPHAGO 01:38:39

+ optimizes a goal with emergent behavior

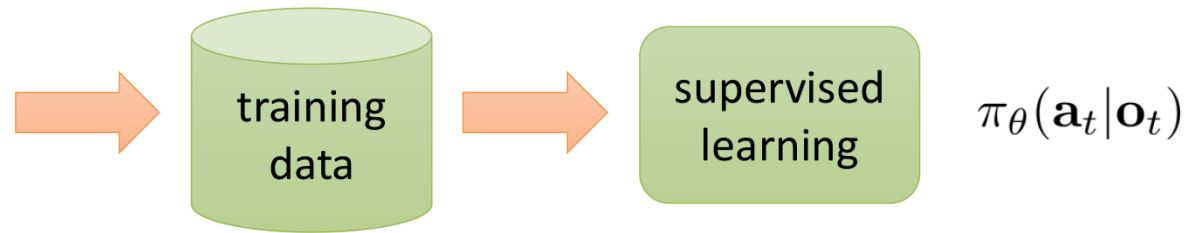
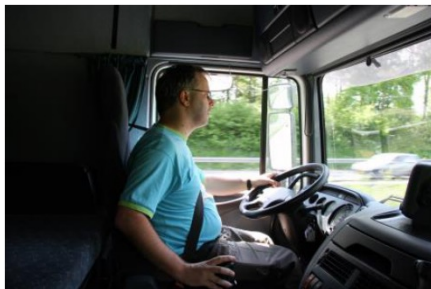
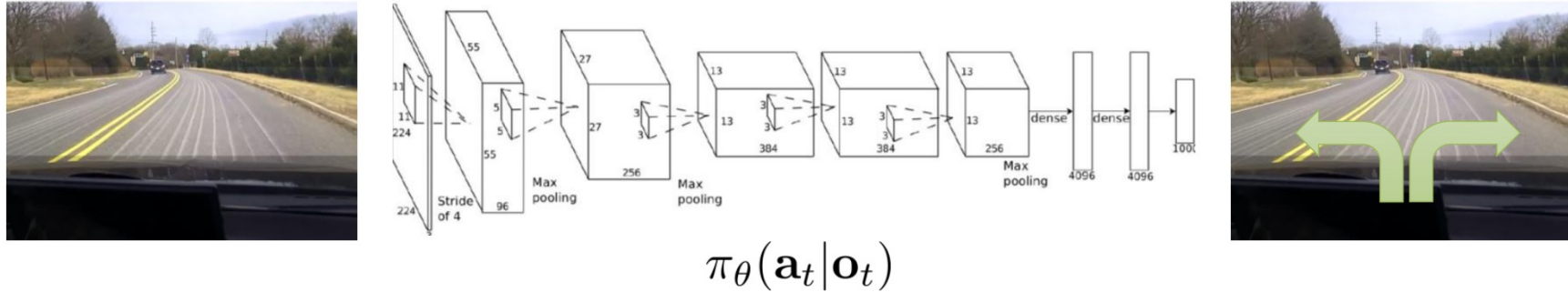
- **but need to figure out how to use at scale!**

Data without optimization
doesn't allow us to solve new
problems in new ways

Imitation learning: learn from demonstrations

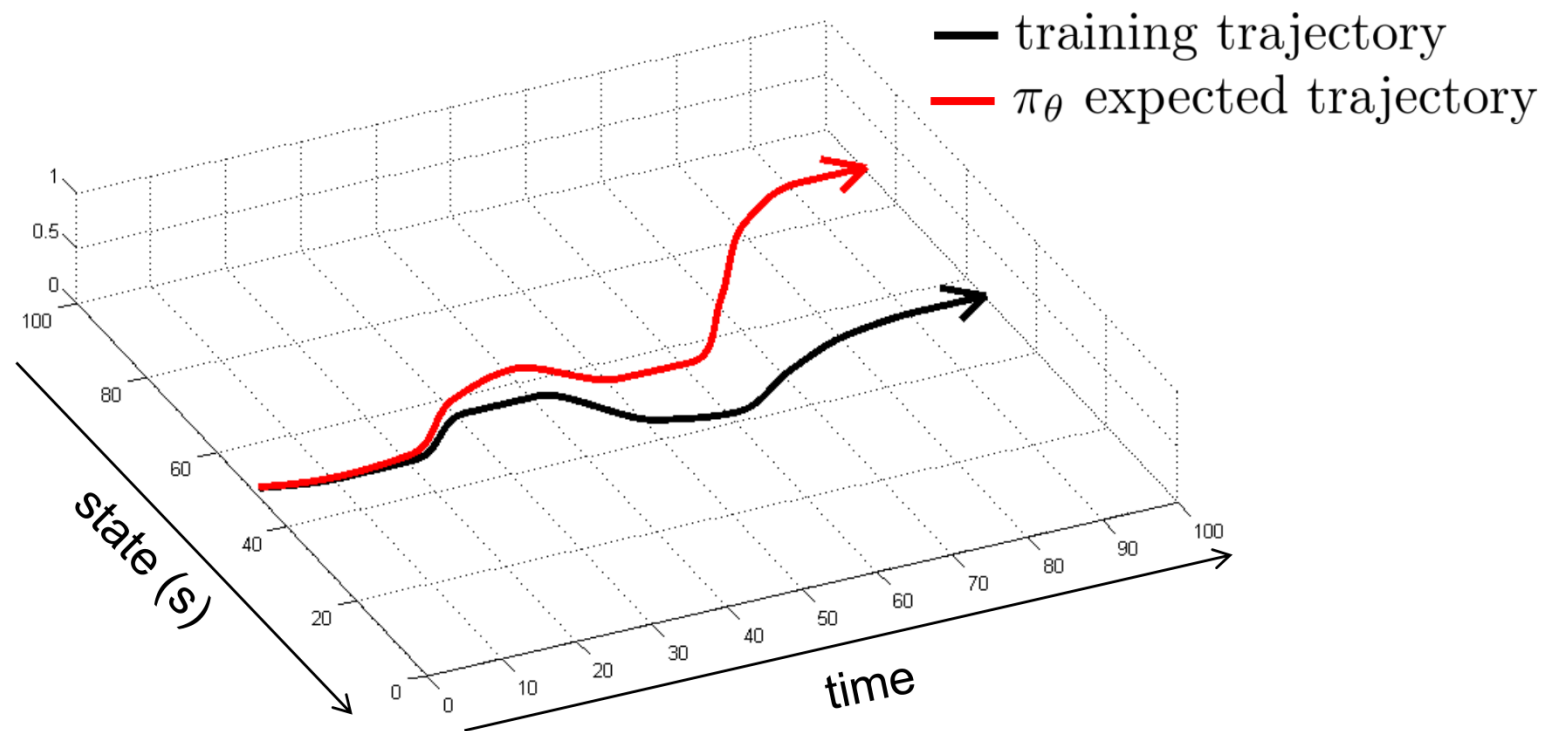
CS285 Lecture 2

Imitation Learning

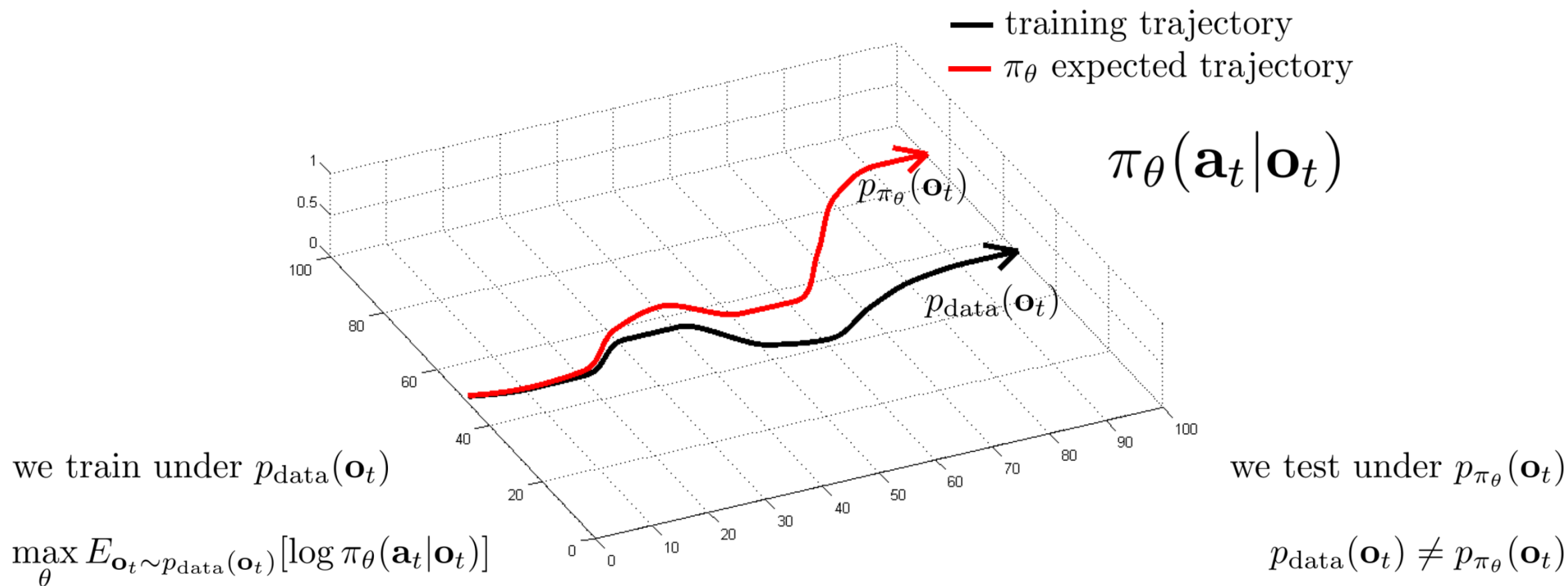


behavioral cloning

No! Does it work?



The distributional shift problem

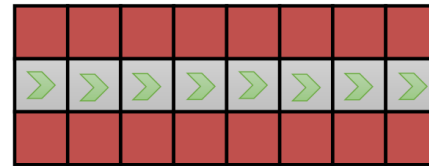
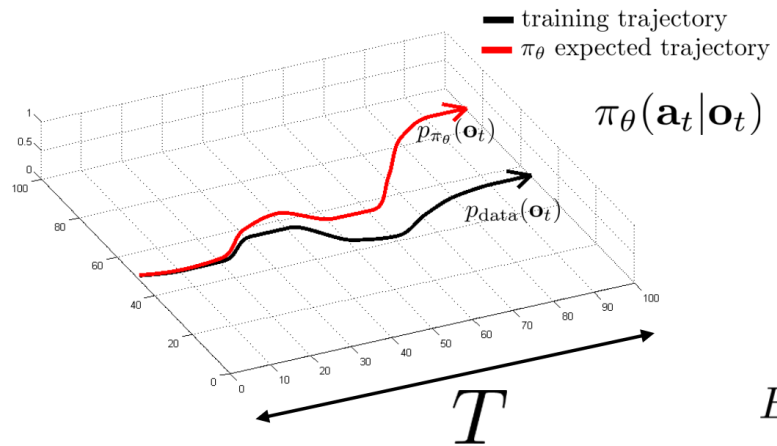


Some analysis

$$c(\mathbf{s}, \mathbf{a}) = \begin{cases} 0 & \text{if } \mathbf{a} = \pi^*(\mathbf{s}) \\ 1 & \text{otherwise} \end{cases}$$

assume: $\pi_\theta(\mathbf{a} \neq \pi^*(\mathbf{s}) | \mathbf{s}) \leq \epsilon$

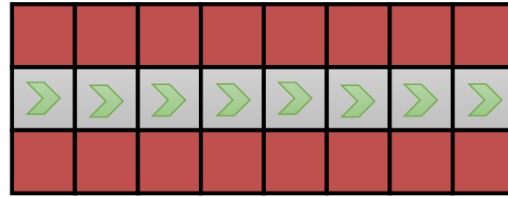
for all $\mathbf{s} \in \mathcal{D}_{\text{train}}$



$$E \left[\sum_t c(\mathbf{s}_t, \mathbf{a}_t) \right] \leq \underbrace{\epsilon T}_{O(\epsilon T^2)} + \underbrace{T \text{ terms, each } O(\epsilon T)}$$

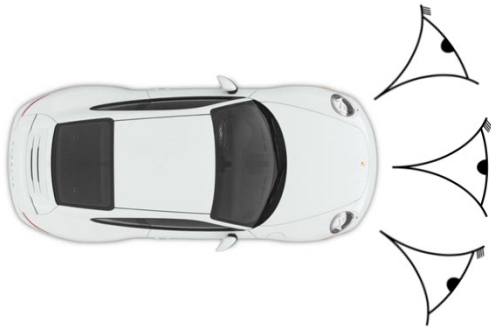


Why is this rather pessimistic?

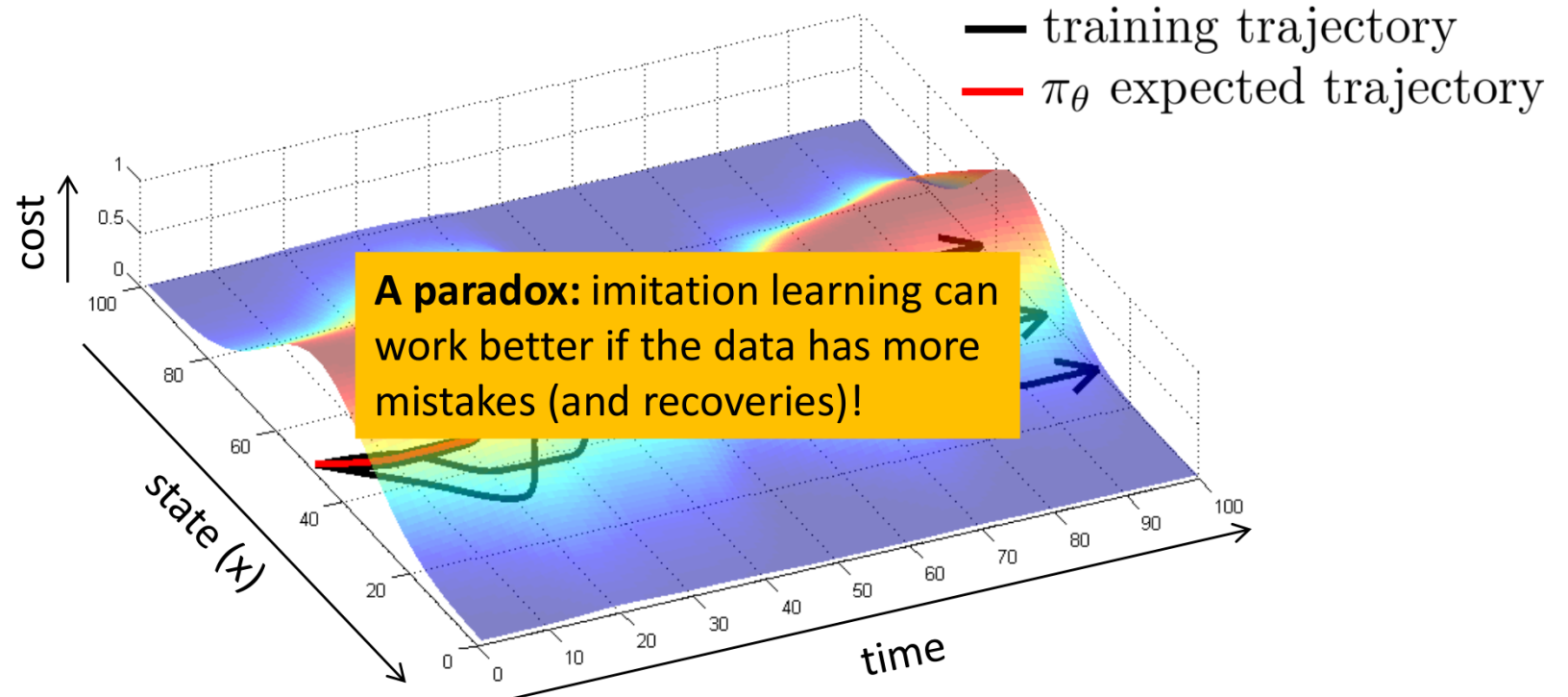


In reality, we can often recover from mistakes

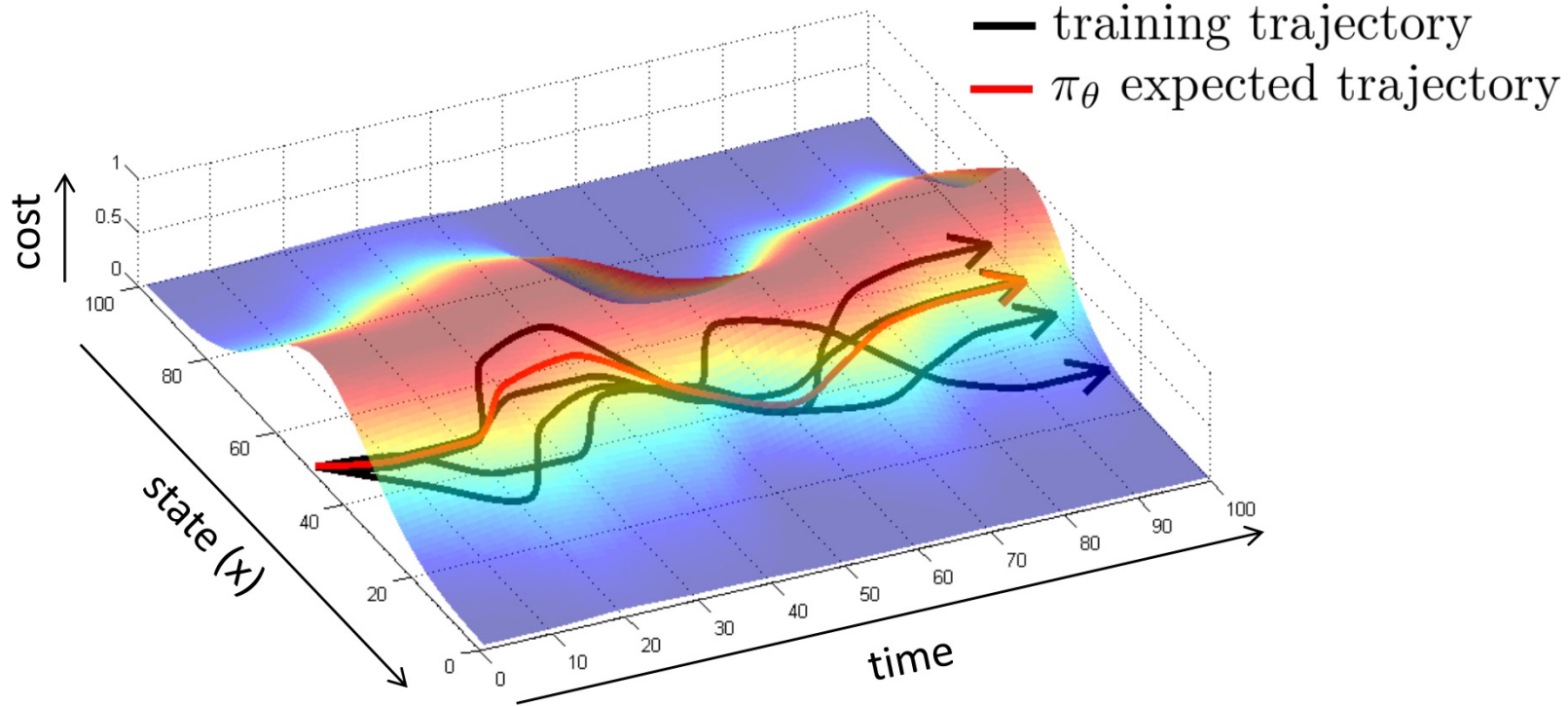
But that doesn't mean that imitation learning will allow us to learn how to do that!



Why does this work?

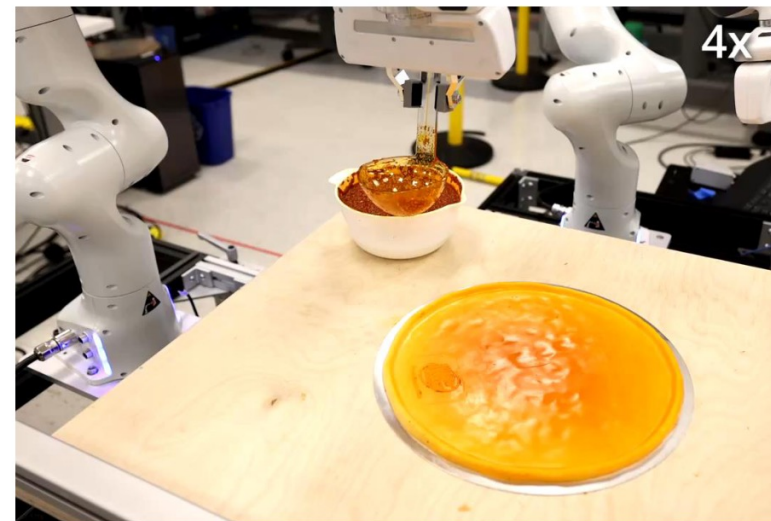
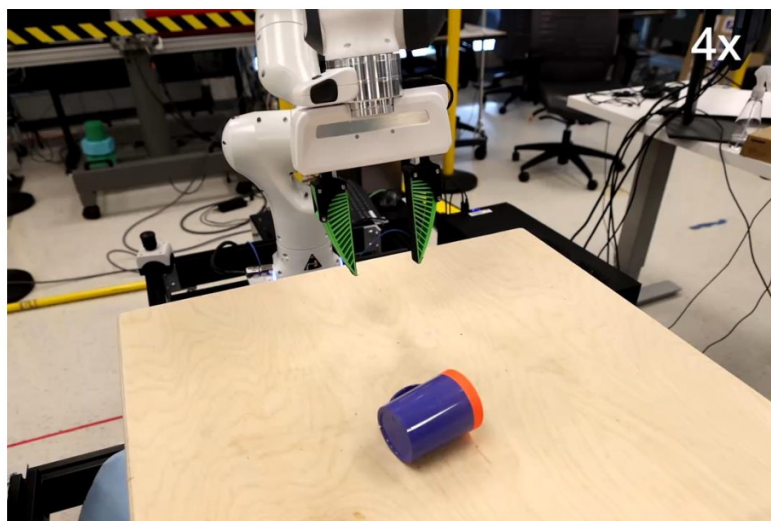
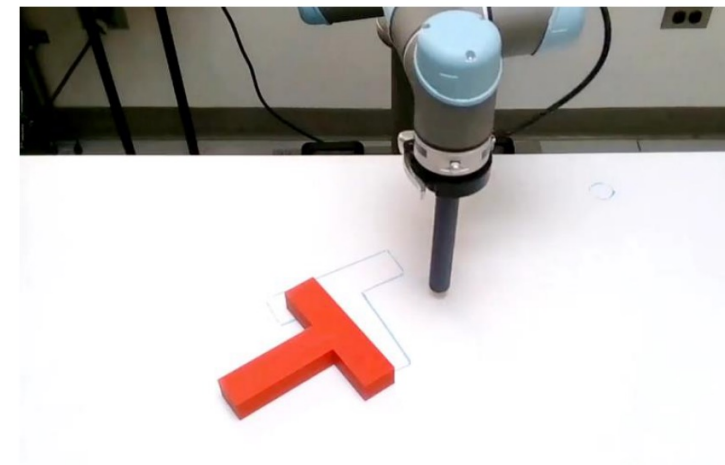
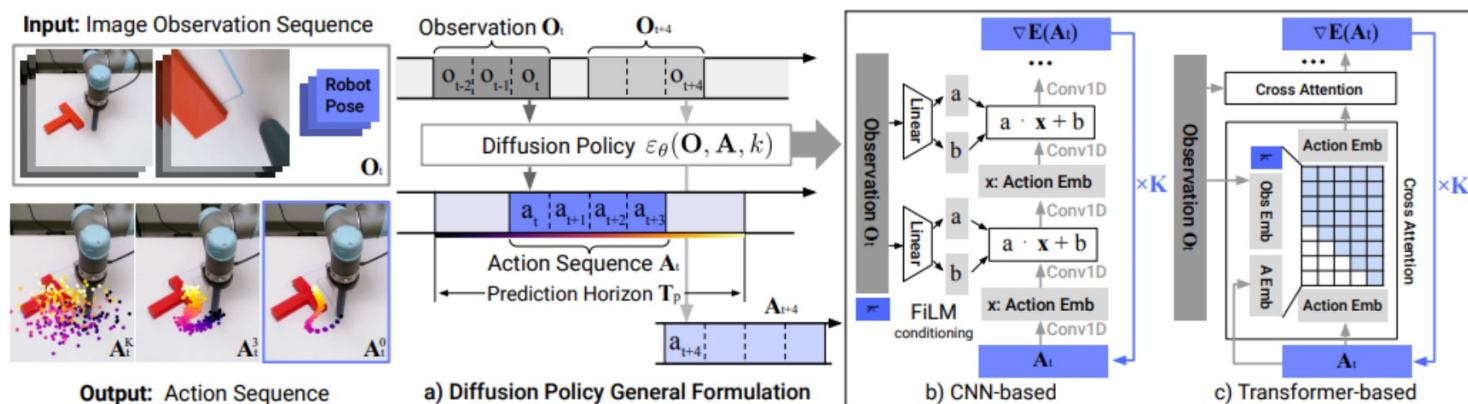


What makes behavioral cloning easy and what makes it hard?



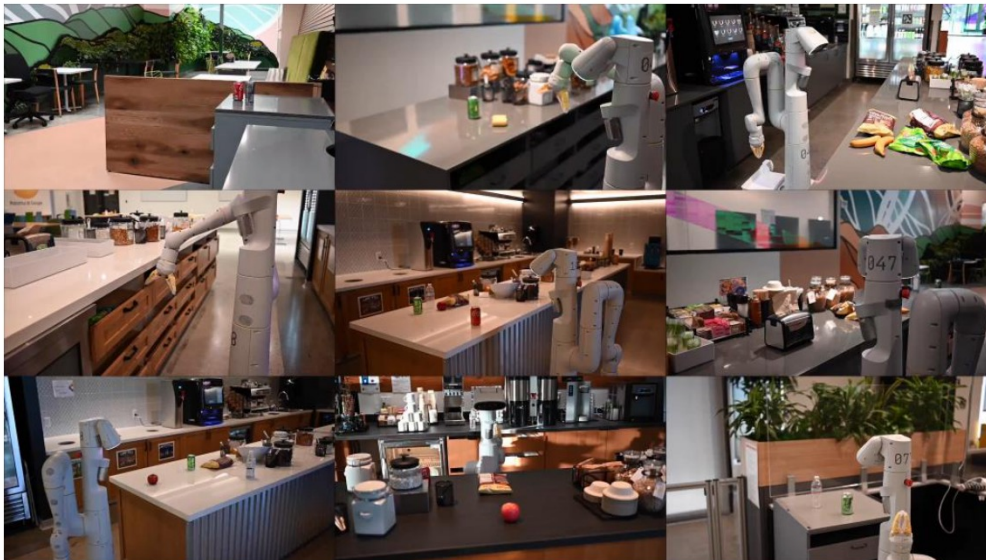
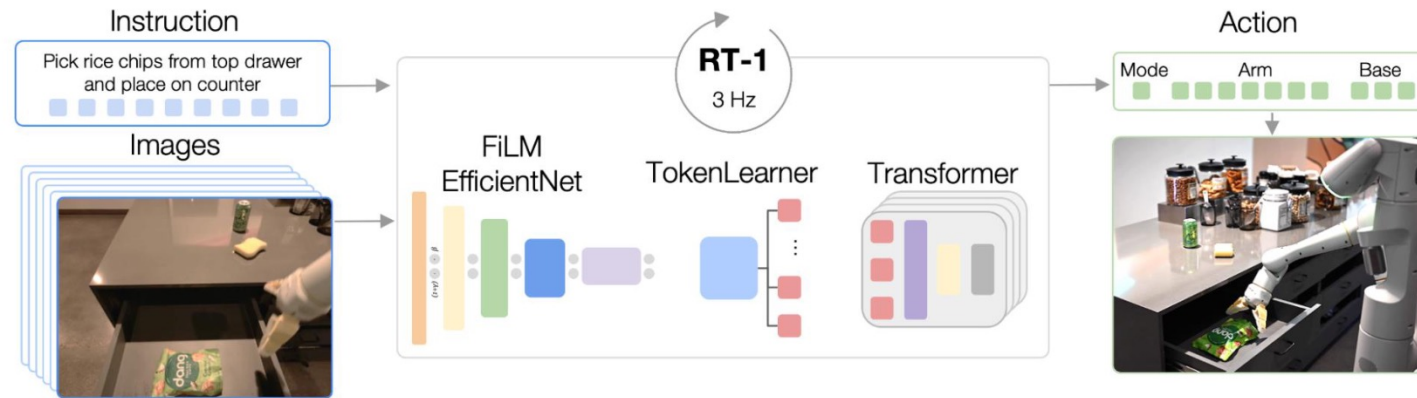
- Intentionally add mistakes and corrections
- The mistakes hurt, but the corrections help, often more than the mistakes hurt
- **Use data augmentation**
- Add some “fake” data that illustrates corrections (e.g., side-facing cameras)

Case study 3: imitation with diffusion models



Chi et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. 2023

Case study 5: imitation with Transformers



RT-1: Robotics Transformer
for Real-World Control at Scale

Robotics at Google

Everyday Robots

Google Research

<https://robotics-transformer.github.io/>

Can we make it work more often?

can we make $p_{\text{data}}(\mathbf{o}_t) = p_{\pi_\theta}(\mathbf{o}_t)$?

idea: instead of being clever about $p_{\pi_\theta}(\mathbf{o}_t)$, be clever about $p_{\text{data}}(\mathbf{o}_t)$!

DAgger: Dataset Aggregation

goal: collect training data from $p_{\pi_\theta}(\mathbf{o}_t)$ instead of $p_{\text{data}}(\mathbf{o}_t)$

how? just run $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

but need labels $\mathbf{a}_t$!

- 
1. train $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ from human data $\mathcal{D} = \{\mathbf{o}_1, \mathbf{a}_1, \dots, \mathbf{o}_N, \mathbf{a}_N\}$
 2. run $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ to get dataset $\mathcal{D}_\pi = \{\mathbf{o}_1, \dots, \mathbf{o}_M\}$
 3. Ask human to label $\mathcal{D}_\pi$ with actions $\mathbf{a}_t$
 4. Aggregate: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_\pi$

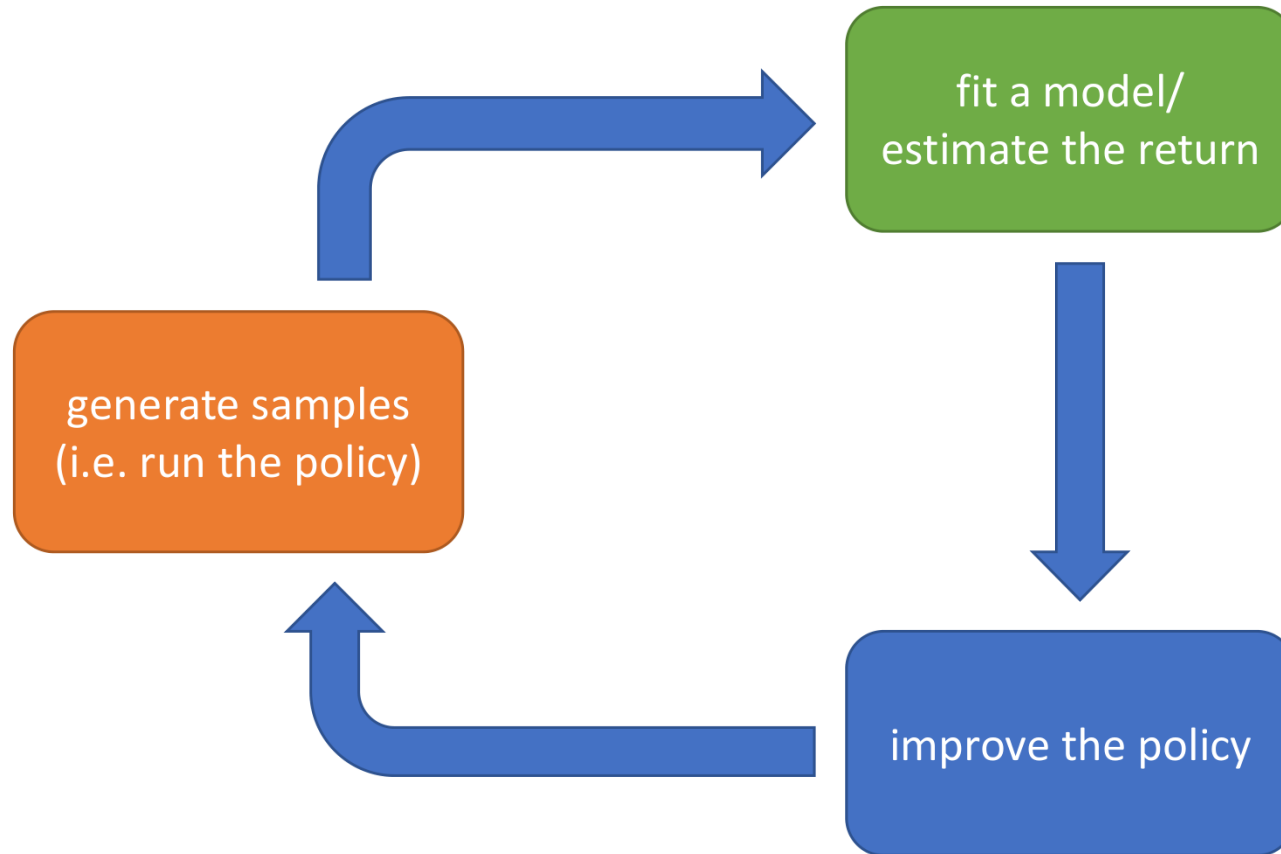
SFT is behavior cloning

- Supervised fine-tuning maximizes $\log \pi(\text{token} \mid \text{prompt, previous tokens})$ on expert demonstrations — exactly behavior cloning on text
- The same failure mode: at test time the model conditions on **its own** previous tokens, never seen in training (exposure bias) $\rightarrow$ errors compound over long generations (the $O(\epsilon T^2)$ argument)
- The same fixes: better and more diverse demonstrations; data that shows recovery from mistakes; label the model's own outputs (DAgger) — RLHF's preference data on model samples is a form of this; or optimize the outcome directly with RL
- Lecture 11's 'Issues with SFT' slide is the LLM version of this slide

The reinforcement learning problem

CS285 Lecture 4

The anatomy of a reinforcement learning algorithm



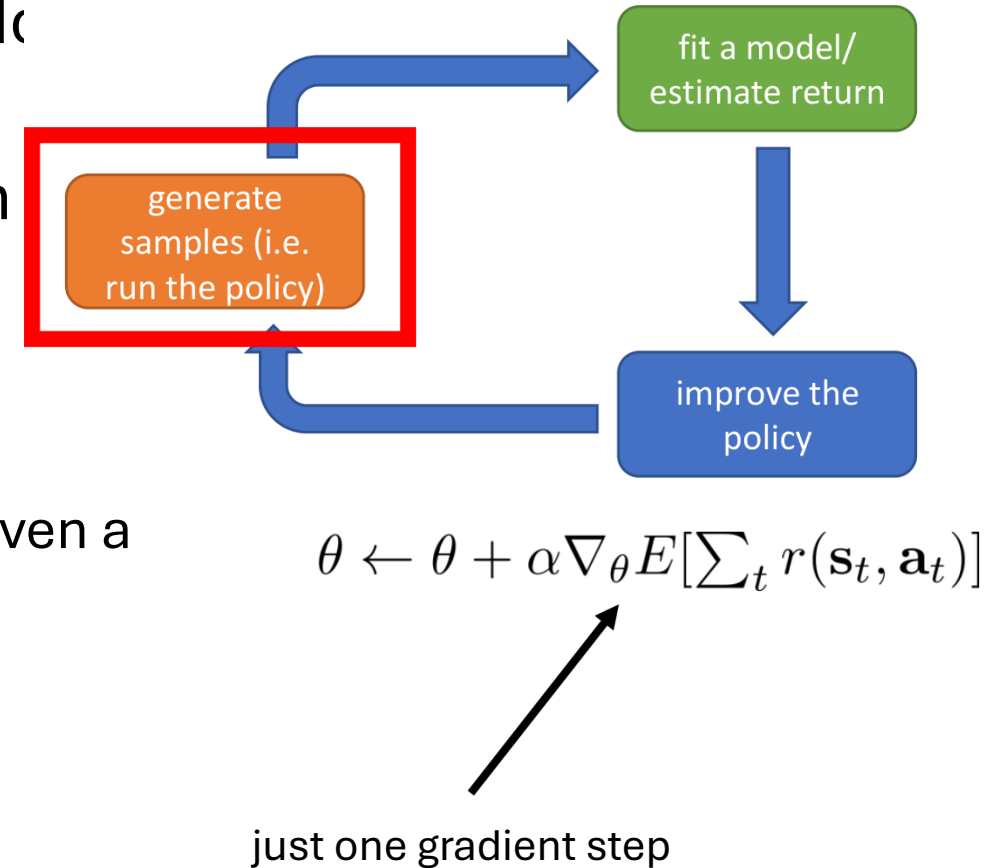
Types of RL algorithms

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

- Policy gradients: directly differentiate the above objective
- Value-based: estimate value function or Q-function of the optimal policy (no explicit policy)
- Actor-critic: estimate value function or Q-function of the current policy, use it to improve policy
- Model-based RL: estimate the transition model, and then...
 - Use it for planning (no explicit policy)
 - Use it to improve a policy
 - Something else

Comparison: sample efficiency

- Sample efficiency = how many samples do we need to get a good policy?
- Most important question: is the algorithm on policy?
 - Off policy: able to improve the policy without generating new samples from that policy
 - On policy: each time the policy is changed, even a little bit, we need to generate new samples



Comparison: stability and ease of use

- Does it converge?
- And if it converges, to what?
- And does it converge every time?

Why is any of this even a question???

- *Supervised learning: almost always gradient descent*
- *Reinforcement learning: often not gradient descent*
- Q-learning: fixed point iteration
- Model-based RL: model is not optimized for expected reward
- *Policy gradient: is gradient descent, but also often the least efficient!*

The language model as a policy

- MDP definitions and the RL objective are in Lecture 11 (slides 27–32); here only the mapping:
state s_t = prompt + tokens so far; **action** a_t = next token; **policy** $\pi_\theta(a_t | s_t)$ = the softmax over the vocabulary
- **Transition**: deterministic — append the token; **trajectory** τ = the whole response (hundreds to thousands of steps)
- **Reward**: usually only at the end — a human-preference reward model (RLHF) or a verifier (RLVR: unit tests, math checkers) — sparse and delayed
- Same for image generators: the denoising steps are the actions (Lecture 10b); for robots: motor commands from camera images

Policy gradients

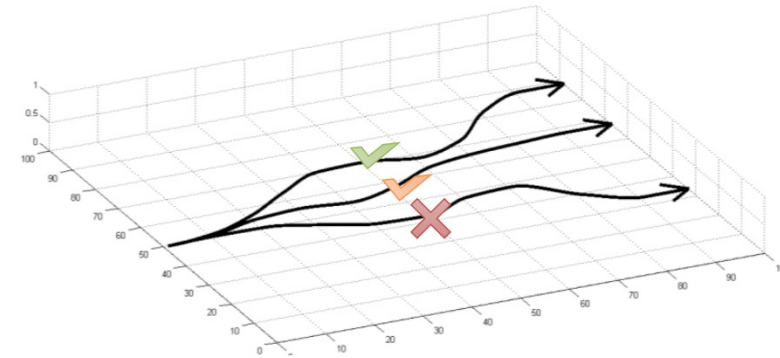
CS285 Lecture 5

Evaluating the objective

$$\theta^* = \arg \max_{\theta} \underbrace{E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]}$$

$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

sum over samples from π_{θ}



Direct policy differentiation

$$\theta^* = \arg \max_{\theta} \underbrace{E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]}$$

a convenient identity

$$\underline{p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)} = p_{\theta}(\tau) \frac{\nabla_{\theta} p_{\theta}(\tau)}{p_{\theta}(\tau)} = \underline{\nabla_{\theta} p_{\theta}(\tau)}$$

$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\underbrace{r(\tau)}_{\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t)} \right] = \int p_{\theta}(\tau) r(\tau) d\tau$$

$$\nabla_{\theta} J(\theta) = \int \underline{\nabla_{\theta} p_{\theta}(\tau)} r(\tau) d\tau = \int \underline{p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)} r(\tau) d\tau = E_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)]$$

Evaluating the policy gradient

$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

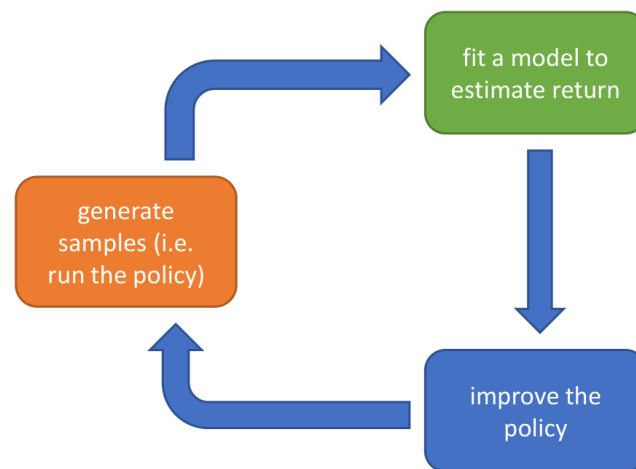
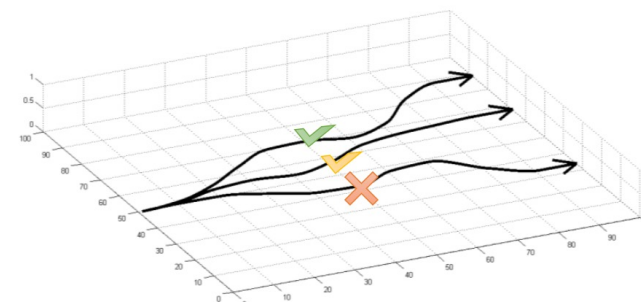
$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

REINFORCE algorithm:

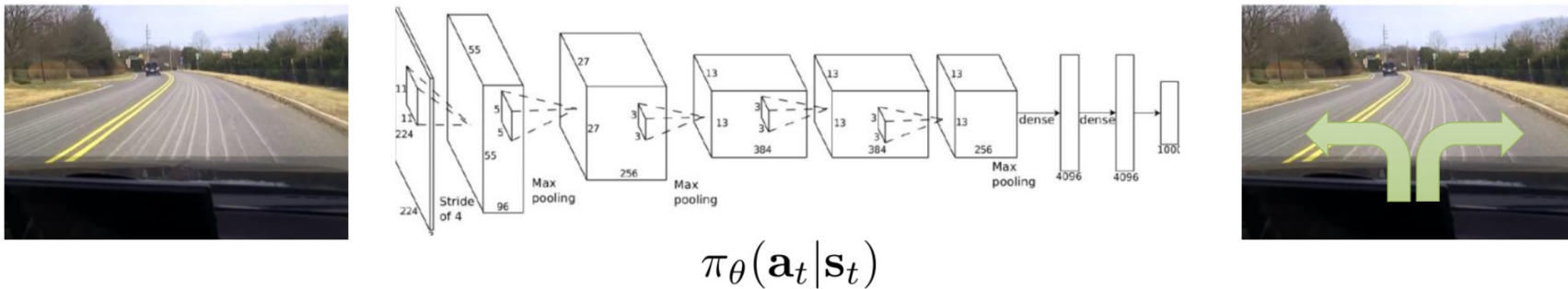
1. sample $\{\tau^i\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run the policy)
2. $\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^i | \mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$



Comparison to maximum likelihood

policy gradient:
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

maximum likelihood:
$$\nabla_{\theta} J_{\text{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right)$$



What did we just do?

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \underbrace{\nabla_{\theta} \log \pi_{\theta}(\tau_i)}_{\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})} r(\tau_i)$$

maximum likelihood:

$$\nabla_{\theta} J_{\text{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau_i)$$

good stuff is made more likely

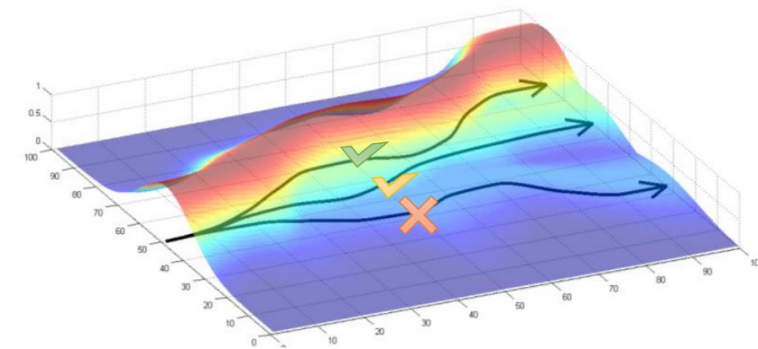
bad stuff is made less likely

simply formalizes the notion of “trial and error”!

REINFORCE algorithm:

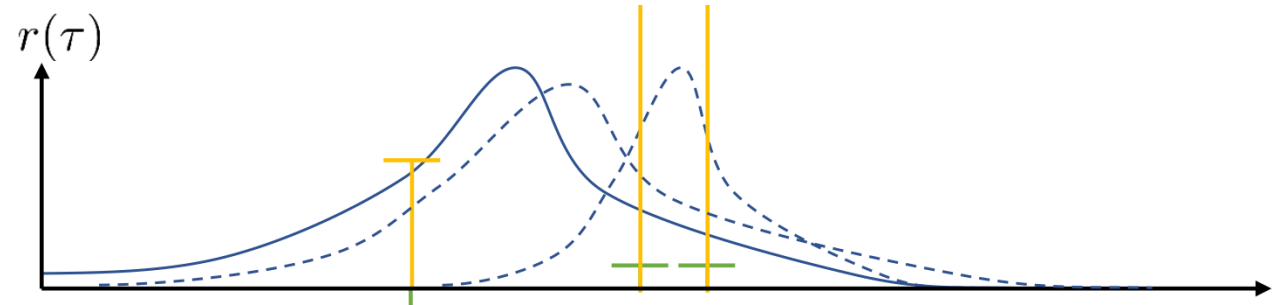


1. sample $\{\tau^i\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run it on the robot)
2. $\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^i | \mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$



What is wrong with the policy gradient?

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau) r(\tau)$$



even worse: what if the two “good” samples have $r(\tau) = 0$?

high variance

Reducing variance

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

Causality: policy at time t' cannot affect reward at time t when $t < t'$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \underbrace{\left(\sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right)}_{\text{“reward to go”}}$$

“reward to go”

Baselines

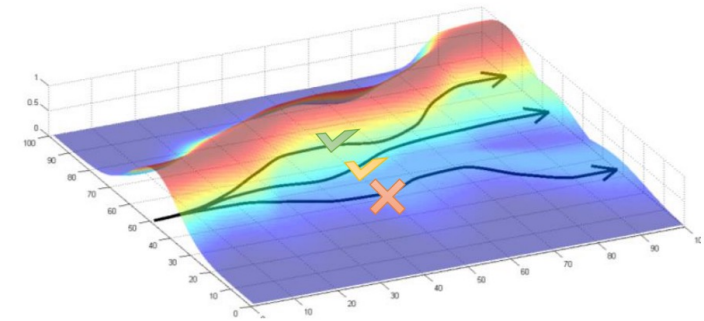
a convenient identity

$$p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) = \nabla_{\theta} p_{\theta}(\tau)$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log p_{\theta}(\tau) [r(\tau) - b]$$

$$b = \frac{1}{N} \sum_{i=1}^N r(\tau)$$

but... are we allowed to do that??



$$E[\nabla_{\theta} \log p_{\theta}(\tau) b] = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) b d\tau = \int \nabla_{\theta} p_{\theta}(\tau) b d\tau = b \nabla_{\theta} \int p_{\theta}(\tau) d\tau = b \nabla_{\theta} 1 = 0$$

subtracting a baseline is unbiased in expectation!

average reward is not the best baseline, but it's pretty good!

Policy gradient is on-policy

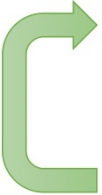
$$\theta^* = \arg \max_{\theta} J(\theta)$$

$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)}[r(\tau)]$$

$$\nabla_{\theta} J(\theta) = \underbrace{E_{\tau \sim p_{\theta}(\tau)}}_{\text{this is trouble...}} [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)]$$

- Neural networks change only a little bit with each gradient step
- On-policy learning can be extremely inefficient!

REINFORCE algorithm:

- 
1. sample $\{\tau^i\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run it on the robot)
 2. $\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^i | \mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
 3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

can't just skip this!

Off-policy learning & importance sampling

$$\theta^* = \arg \max_{\theta} J(\theta)$$

$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)}[r(\tau)]$$

what if we don't have samples from $p_{\theta}(\tau)$?

(we have samples from some $\bar{p}(\tau)$ instead)

$$J(\theta) = E_{\tau \sim \bar{p}(\tau)} \left[\frac{p_{\theta}(\tau)}{\bar{p}(\tau)} r(\tau) \right]$$

$$p_{\theta}(\tau) = p(\mathbf{s}_1) \prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\frac{p_{\theta}(\tau)}{\bar{p}(\tau)} = \frac{\cancel{p(\mathbf{s}_1)} \prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \cancel{p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}}{\cancel{p(\mathbf{s}_1)} \prod_{t=1}^T \bar{\pi}(\mathbf{a}_t | \mathbf{s}_t) \cancel{p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}} = \frac{\prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)}{\prod_{t=1}^T \bar{\pi}(\mathbf{a}_t | \mathbf{s}_t)}$$

importance sampling

$$\begin{aligned} E_{x \sim p(x)}[f(x)] &= \int p(x) f(x) dx \\ &= \int \frac{q(x)}{q(x)} p(x) f(x) dx \\ &= \int q(x) \frac{p(x)}{q(x)} f(x) dx \\ &= E_{x \sim q(x)} \left[\frac{p(x)}{q(x)} f(x) \right] \end{aligned}$$

The off-policy policy gradient

$$\theta^* = \arg \max_{\theta} J(\theta) \quad J(\theta) = E_{\tau \sim p_{\theta}(\tau)}[r(\tau)]$$

$$\nabla_{\theta'} J(\theta') = E_{\tau \sim p_{\theta}(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \nabla_{\theta'} \log \pi_{\theta'}(\tau) r(\tau) \right] \quad \text{when } \theta \neq \theta'$$

$$\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} = \frac{\prod_{t=1}^T \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)}$$

$$= E_{\tau \sim p_{\theta}(\tau)} \left[\left(\prod_{t=1}^T \frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \right) \left(\sum_{t=1}^T \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \right) \right] \text{ what about causality?}$$

$$= E_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t) \underbrace{\left(\prod_{t'=1}^t \frac{\pi_{\theta'}(\mathbf{a}_{t'} | \mathbf{s}_{t'})}{\pi_{\theta}(\mathbf{a}_{t'} | \mathbf{s}_{t'})} \right)}_{\text{future actions don't affect current weight}} \left(\sum_{t'=t}^T r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \right) \underbrace{\left(\prod_{t''=t}^{t'} \frac{\pi_{\theta'}(\mathbf{a}_{t''} | \mathbf{s}_{t''})}{\pi_{\theta}(\mathbf{a}_{t''} | \mathbf{s}_{t''})} \right)}_{\text{if we ignore this, we get a policy iteration algorithm (more on this in a later lecture)}} \right]$$

future actions don't affect current weight

if we ignore this, we get
a policy iteration algorithm
(more on this in a later lecture)

Policy gradient in practice

- Remember that the gradient has high variance
- This isn't the same as supervised learning!
- Gradients will be really noisy!
- Consider using much larger batches
- Tweaking learning rates is very hard
- Adaptive step size rules like ADAM can be OK-ish
- We'll learn about policy gradient-specific learning rate adjustment methods later!

Actor-critic and trust regions

CS285 Lectures 6 and 9

Improving the policy gradient

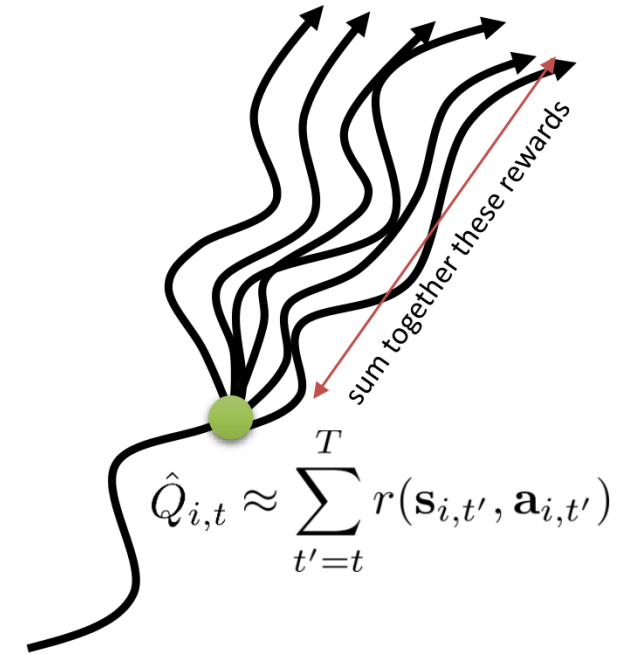
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \underbrace{\left(\sum_{t'=1}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right)}_{\text{“reward to go”}}$$

$\hat{Q}_{i,t}$: estimate of expected reward if we take action $\mathbf{a}_{i,t}$ in state $\mathbf{s}_{i,t}$

can we get a better estimate?

$Q(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^T E_{\pi_{\theta}} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$: true *expected* reward-to-go

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) Q(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$



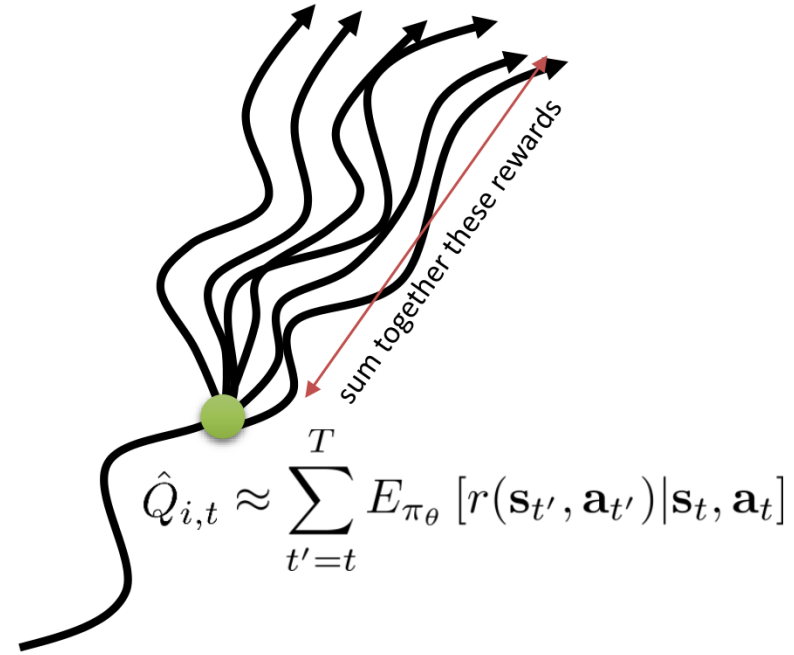
What about the baseline?

$Q(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^T E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$: true *expected* reward-to-go

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) (Q(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) - V(\mathbf{s}_{i,t}))$$

$$b_t = \frac{1}{N} \sum_i Q(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \quad \text{average what?}$$

$$V(\mathbf{s}_t) = E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} [Q(\mathbf{s}_t, \mathbf{a}_t)]$$



State & state-action value functions

$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^T E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$: total reward from taking $\mathbf{a}_t$ in $\mathbf{s}_t$

$V^\pi(\mathbf{s}_t) = E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} [Q^\pi(\mathbf{s}_t, \mathbf{a}_t)]$: total reward from $\mathbf{s}_t$

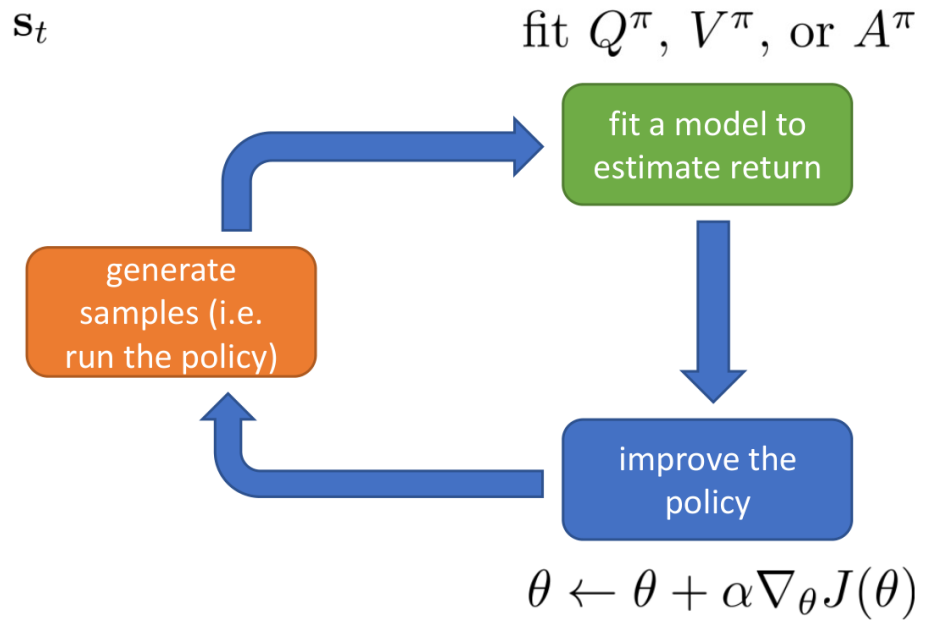
$A^\pi(\mathbf{s}_t, \mathbf{a}_t) = Q^\pi(\mathbf{s}_t, \mathbf{a}_t) - V^\pi(\mathbf{s}_t)$: how much better $\mathbf{a}_t$ is

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) A^\pi(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

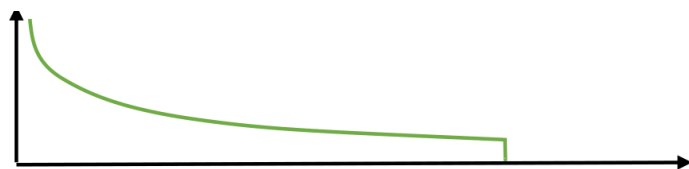
the better this estimate, the lower the variance

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \left(\sum_{t'=1}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) - b \right)$$

unbiased, but high variance single-sample estimate



Generalized advantage estimation



Do we have to choose just one n ?

Cut everywhere all at once!

$$\hat{A}_n^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_\phi^\pi(\mathbf{s}_t) + \gamma^n \hat{V}_\phi^\pi(\mathbf{s}_{t+n})$$

$$\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{n=1}^{\infty} w_n \hat{A}_n^\pi(\mathbf{s}_t, \mathbf{a}_t)$$

weighted combination of n -step returns

$$w_n \propto \lambda^{n-1} \quad \text{exponential falloff}$$

How to weight? Mostly prefer cutting earlier (less variance)

$$\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma((1 - \lambda)\hat{V}_\phi^\pi(\mathbf{s}_{t+1}) + \lambda(r(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) + \gamma((1 - \lambda)\hat{V}_\phi^\pi(\mathbf{s}_{t+2}) + \lambda r(\mathbf{s}_{t+2}, \mathbf{a}_{t+2}) + \dots))$$

$$\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t'} \quad \delta_{t'} = r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) + \gamma\hat{V}_\phi^\pi(\mathbf{s}_{t'+1}) - \hat{V}_\phi^\pi(\mathbf{s}_{t'})$$

← similar effect as discount!

option 1:
$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \left(\sum_{t'=t}^T \gamma^{t'-t} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right)$$

remember this? discount =
variance reduction!

Ignoring distribution mismatch?

$$\sum_t E_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \stackrel{?}{\approx} \underbrace{\sum_t E_{\mathbf{s}_t \sim p_{\theta}(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]}_{\bar{A}(\theta')}$$

why do we want this to be true?

$$J(\theta') - J(\theta) \approx \bar{A}(\theta') \quad \theta' \leftarrow \arg \max_{\theta'} \bar{A}(\theta)$$

2. Use $\hat{A}^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ to get *improved* policy π'

is it true? and when?

Claim: $p_{\theta}(\mathbf{s}_t)$ is *close* to $p_{\theta'}(\mathbf{s}_t)$ when π_{θ} is *close* to $\pi_{\theta'}$

A more convenient bound

Claim: $p_\theta(\mathbf{s}_t)$ is *close* to $p_{\theta'}(\mathbf{s}_t)$ when π_θ is *close* to $\pi_{\theta'}$

$\pi_{\theta'}$ is *close* to π_θ if $|\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) - \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)| \leq \epsilon$ for all $\mathbf{s}_t$

$$|p_{\theta'}(\mathbf{s}_t) - p_\theta(\mathbf{s}_t)| \leq 2\epsilon t$$

a more convenient bound: $|\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) - \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)| \leq \sqrt{\frac{1}{2} D_{\text{KL}}(\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) \parallel \pi_\theta(\mathbf{a}_t|\mathbf{s}_t))}$

$D_{\text{KL}}(\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) \parallel \pi_\theta(\mathbf{a}_t|\mathbf{s}_t))$ bounds state marginal difference

$$D_{\text{KL}}(p_1(x) \parallel p_2(x)) = E_{x \sim p_1(x)} \left[\log \frac{p_1(x)}{p_2(x)} \right]$$

KL divergence has some very convenient properties that make it much easier to approximate!

Practical methods and notes

- Natural policy gradient
- Generally a good choice to stabilize policy gradient training
- See this paper for details:
- Peters, Schaal. Reinforcement learning of motor skills with policy gradients.
- Practical implementation: requires efficient Fisher-vector products, a bit non-trivial to do without computing the full matrix
- See: Schulman et al. Trust region policy optimization
- Trust region policy optimization
- Just use the IS objective directly
- Use regularization to stay close to old policy
- See: Proximal policy optimization

$$\theta' = \theta + \alpha \mathbf{F}^{-1} \nabla_{\theta} J(\theta)$$

$$\alpha = \sqrt{\frac{2\epsilon}{\nabla_{\theta} J(\theta)^T \mathbf{F} \nabla_{\theta} J(\theta)}}$$

From REINFORCE to PPO to GRPO

- **REINFORCE**: $\nabla J = E[\sum_t \nabla \log \pi(a_t|s_t) \cdot R]$ — unbiased, high variance; reward-to-go and a baseline $b(s)$ fix the variance
- **Actor-critic**: learn $V(s)$ as the baseline $\rightarrow$ advantage $A = R - V$; GAE trades bias for variance
- **PPO** (Lecture 11): the importance ratio $\pi_\theta / \pi_{\text{old}}$ with **clipping** (a cheap trust region), a value network (critic), and a KL penalty to the reference model
- **GRPO** (Lecture 12): sample G responses per prompt; advantage $= (r - \text{mean}_G) / \text{std}_G$ — the **group replaces the critic**; ratio, clipping and KL stay; with a verifier as reward this is RLVR (DeepSeek-R1)

Robots and LLMs learn the same way in 2025–26

- **Vision-language-action models** = behavior cloning with a VLM backbone: RT-2 (Google DeepMind, 2023) emits actions as text tokens; **$\pi 0$** (Physical Intelligence, Oct 2024) adds a flow-matching action head (Lecture 10b); **$\pi 0.5$** (Apr 2025) targets open-world generalization; **Gemini Robotics** (Mar 2025)
- Then the same second stage as LLMs: RL fine-tuning on outcomes (success detectors) or human corrections (DAgger-style)
- LLMs: SFT (imitation) $\rightarrow$ RLHF (learned reward) $\rightarrow$ RLVR (verifier reward, Lecture 12); image/video generators: DDPO / Flow-GRPO (Lecture 10b)
- Levine's summary (Lecture 1): data teaches what the world looks like; optimization is what solves new problems

Summary

- Imitation learning is supervised learning on demonstrations; it fails through distribution shift ($O(\epsilon T^2)$) unless the data shows recovery — SFT inherits both
- RL optimizes expected return under the policy's own trajectory distribution; policy gradients are maximum likelihood weighted by reward
- Variance reduction (reward-to-go, baselines, critics) and trust regions (importance ratios + clipping/KL) turn REINFORCE into PPO; dropping the critic for group statistics gives GRPO
- Next: Lecture 11 (RLHF, PPO, DPO) and Lecture 12 (GRPO, reasoning, RLVR)